The Assistant on the Search Page

Write-up of WeRoad's take-home case · Homayoun Afshari · github.com/homayoun-afshari/weroad-case

The prototype is a trip search page with a chat assistant beside it (the screen is drawn in `UI_LAYOUT.svg`; like the other figures, it shows my plan beside what was built). The page opens with a search already applied ("northern lights"). The assistant turns what the user says into a new search through the same search function the page uses, so the results list updates, and it answers questions on departures, itineraries and FAQs with tools of its own. How to run it is in the `README.md`.

My Approach

I had two and a half days: a day to design, a day to build, and half a day to review and explain.

Day 1: design (`DESIGN.md`). Before any code, I wrote down my assumptions, the business rules, how the prototype works (screen, state, search, agent, errors) and 16 design choices, each with the options I compared and the reason for my pick. The test cases came from the rules and the data's traps.

Day 2: build (`BUILD.md`). The build went in seven blocks, each on its own branch: the search function first (since it holds most of the rules), then the page, the agent, the tools, the failure paths, an eval (offline cases in code and live conversations with the model, in `EVALS.md`), and the fixes. A coding agent wrote the code and tests to the design; I reviewed and tested each step, and logged 36 issues with how each was settled. The result is a Next.js app with the Vercel AI SDK on Claude Haiku 5.5:

- **The search function:** a rule-based translator reads destinations, trip types and prices from the text, a mock of WeRoad's Search API filters the 50 trips, and a suggester steps in when nothing matches. It is described in `SEARCH_FUNCTION.md` and drawn in `SEARCH_FUNCTION.svg`.
- **The agent:** one bounded tool-calling loop with five tools: `search_trips`, `get_departures`, `get_itinerary`, `get_faq` and `hand_over`, each with its schemas and an example in `AGENT_GRAPH.md` and drawn in `AGENT_GRAPH.svg`. The agent's system prompt is explained in `PROMPT.md`.

Last half-day: review. I checked my decisions against the brief, went back over the issues, wrote the other documents and this write-up, and prepared the walkthrough.

Decisions Made

Assumptions

- **The page and the chat work in harmony; neither replaces the other.** There is one search state: the assistant changes the search request, and the page shows what the search function returns, so the two cannot disagree. Each user message also carries what the user did on the page and the page as it is now, so the agent never asks what the page already shows.
- **The page never depends on the chat.** A search makes no model call, so a model failure, its latency or its cost never blocks the page. The chat opens on a button, and its first message is written by code from the page state.
- **Business rules live in code, with tests, never in the prompt.** The tools apply them, and the agent repeats only what they return.

Design choices

- **One rule-based search function, two callers.** The search button and `search_trips` call the same function. The tool forms the complete new request (keeping, changing or dropping parts of the current one) and runs the search in the same step, so the agent's answer and the page come from the same results. Rules, not a model, read the search text, so a search is free, fast and works when the model

doesn't. The cost: rules understand fewer phrasings, and the agent covers that by setting values directly.

- **Narrowing through search.** "Only the ones with a cooking class" adds "cooking" to the search text, instead of the agent picking a subset of the results. This way, the search controls can also reproduce the results if needed, and filtering stays with the Search API. The limit: it narrows by words, not by meaning.
- **Empty results are replaced, by code.** When a search finds nothing, a suggester loosens it one value at a time in a fixed order (dates, price, duration, the other filters, the text last), within a fixed number of calls. The closest trips replace the empty list, each tagged with how it differs ("€1,120 · above your €800").
- **Two histories, on the client:** the page history for undo, and the chat history, sent with every turn. The server needs no database.

Business rules in code

- **Bookability comes from the sales status, never from seats left**: 129 NOT_AVAILABLE departures still show seats. Four statuses are bookable, two of them flagged as not yet confirmed; the waiting list is offered only beside a bookable alternative; past departures are dropped.
- **A trip matches the filters only through one bookable departure that meets them all**, and its price is the cheapest such departure.
- **A date window matches departures fully inside it**; when none does, the nearest ones before and after are given.
- **FAQ answers come only from retrieved articles**; otherwise the agent says so and offers a human.

What I Left Out

- **A real hand-over.** The agent offers a human when it can't answer, but yes just ends the chat.
- **Narrowing by meaning**, which needs a tool that picks a subset of the results.
- **The tools the data can't support:** website pages, promotions and flight prices.
- **A real RAG.** Picking FAQs by their destination field works for 50 English articles, not for a multilingual help center.
- **Production plumbing:** the histories are lost on reload; no cap on the history sent to the model; no tracing; the controls lock during a turn.

Next Steps

1. **Evaluate, then A/B test** on the brief's metrics (the share of searches that open the assistant, their conversion, the undo rate, the zero-result share, latency), with product clicks as the guardrail. The prototype has no booking, so conversion can only be measured there. `METRICS.md` says where in the code each metric would be counted.
2. **Suggestions when results are few.** Today they appear only when there are none. If users often get one or two results and leave, add suggestions below a threshold.
3. **Store the errors and the hand-overs on the server,** to see what fails and how often, and to grow the eval from real cases.

A suggestion for WeRoad: a fuller Search API

Based on my current knowledge, the WeRoad's Search API takes a search request and returns an ordered list of trips, which is why my prototype wraps it in its own translator and suggester. I suggest a fuller Search API: given a search request, it would return the trips that match and also a few suggestions (the nearest trips when the request is too strict), ranked among the results, each with what was loosened to find it. In my opinion every page would gain from it, not only the chat, since most users never open the chat, and the site, the apps and the chatbot would then suggest the same trips.